SECTION 4 · CHEAT SHEET

Plan and guard rails

Every prompt, command, shortcut and setting from this section, lesson by lesson. Prompts are word for word, as typed on screen.

4.1 Section 4 introduction

Concept lesson: nothing to type, run or set.

4.2 Plan with the agent

COMMAND

```
/plan
```

Type a slash, pick **plan**, and Enter turns it into a chip. The agent researches and writes a plan; it changes no files.

PROMPT

```
Plan the cafe's website from the owner's brief in baseline/new_outlet_brief.md:
home, about, menu, cart and a demo checkout with a pickup time. Use plain HTML, CSS
and JavaScript in a site folder, with the menu from baseline/menu.csv. Explain what
happens in an order in plain business words. Don't build anything yet.
```

The agent reads the brief, the menu and the profiles first, then answers with the order in five plain steps and an **Implementation Plan**.

SETTING

Open the plan › the side panel

The goal, the order as a diagram, the questions it needs answered, every proposed file and each page in detail.

RESULT

Proceed (Ctrl+Enter)

Approves the plan and starts the build. Read the plan first: a plan is far cheaper to correct than a website.

4.3 Let the agent question your plan

COMMAND

```
/grill-me
```

The agent interviews you about the plan, one question at a time, each with answers to pick and its recommendation first.

PROMPT

```
Question me about the order flow in this plan, like an empty cart or a pickup time
that has already passed, then update the plan. Don't build anything yet.
```

Sent in the same conversation as the plan.

SETTING

Question card › pick an answer › Submit

Five questions here: past pickup times and opening hours, the empty cart, the customer details (**name only**, our own choice), the order ticket after a refresh, and the outlets.

SETTING

Question card › the last field (Other) › type your own answer › Submit

When no answer fits: Both outlets selectable, Koramangala by default. Mark Indiranagar as opening soon, with no dates.

RESULT

The plan is updated

Every answer is written into the plan, decision by decision. No website files yet.

4.4 House rules

PROMPT

```
Write the cafe's house rules for every agent in this project into AGENTS.md:
friendly, plain words for customers; prices in USD with two decimals, as in
baseline/menu.csv; the checkout is a demo, so never ask for card or payment
details; no years or calendar dates anywhere on the website; never put passwords,
keys or tokens in any file. Also add .agents/rules/site-look.md that applies only
to files in the site folder: use the fonts in brand/fonts.md and the colours of the
printed menu in source/KGP_Vibes_Cafe_Menu.pdf.
```

Sent in a new conversation in the cafe project.

SHORTCUT

`The stop button (bottom right of the message box)`

The agent dug into its own program for the rule format. Stop it, then steer it in one exact sentence.

PROMPT

```
You don't need to look inside the program. A rule file starts with frontmatter, for
example trigger: glob and globs: site/**. Please write both files now.
```

Both files are written: **AGENTS.md** (always on) and **.agents/rules/site-look.md** (only for files in site/).

SETTING

Sidebar: the menu beside the project › Project Settings › Customizations › Rules › Show breakdowns

Lists every rules file loaded for the project. A glob rule is not listed: it joins in only for the files it names.

4.5 Keep going until it works

COMMAND

```
/goal
```

Sets what done looks like. The agent keeps working, testing and fixing until it gets there.

PROMPT

```
Build the website from the plan above, which I approve. The goal is reached when a
demo order placed in the browser ends with an order number.
```

Sent in the planning conversation, so the whole plan is in front of the agent.

SETTING

Overview panel › Goals

The goal with a spinner while it is open, and a tick once it is reached.

RESULT

All tests passed in a browser

The agent wrote and ran its own browser test of a whole order, and checked the house rules too: no dates, USD with two decimals.

PROMPT

```
Start a local server for the site folder and tell me the address, so I can place a
demo order myself.
```

The site runs at **localhost:8000**. The link opens it in the side panel.

RESULT

A demo order with an order number

Cold brew and cappuccino, total **$7.20**, no card asked for, and an order number on the ticket.

4.6 Effort for a hard task

SETTING

Message box › the model picker › Gemini 4 Argon › High

Argon is the deepest thinker and has its own usage limit: keep it for hard problems. Effort (Low, Medium, High) is how long the model may think before it acts.

PROMPT

```
The pickup time on the cart page must be right at every time of day. Right now,
late at night, every quick button gives the same 8:30 AM. Offer pickup times as 15-
minute slots from 8:00 AM to 9:45 PM, never in the past and at least 15 minutes
from now. When the cafe is closed, say so and start from the next opening time.
Hide quick buttons that would land after closing. The ticket must show the time the
customer picked, and the cart totals must stay correct when quantities change. Test
it at several times of day before you finish.
```

Sent in a new conversation. Argon worked about twelve minutes and tested the page at eight times of day.

RESULT

Pickup times as 15-minute slots

At night the cart says the cafe is closed and starts at 8:00 AM; +30 min gives 8:15, +45 min gives 8:30; the ticket shows the time picked.

SETTING

Model picker › Gemini 3.8 Flash › High

Back to the everyday model once the hard part is done.

4.7 Learn from corrections

PROMPT

```
The closed notice on the cart page is too long. Make it: We're closed right now.
Pickup starts at 8:00 AM. Use the next opening time there. Keep every message a
customer sees short and plain, one sentence where you can.
```

Sent in the same conversation. The agent shortened the notice and the other customer texts on the cart page.

COMMAND

```
/learn from my correction about short customer messages
```

The agent changes nothing yet: it writes a **Learning Proposal** with the kind of lesson, the reason and the exact change.

RESULT

Proceed

Approves the proposal: one new line in rule 1 of **AGENTS.md**, so every agent in the project keeps customer messages short from now on.

4.8 Permission presets

SETTING

Message box › the shield

Request Review: every command needs approval. **Default**: asks before sensitive operations. **Auto (Preview)**: an automatic check decides. **Turbo**: every command runs without asking.

SETTING

Settings › General › Permission Preset

The global preset. Every project uses it unless the project sets its own.

SETTING

Settings › Projects › kgp-vibes-cafe-main › Permission Preset › Default

The project's own preset, changed from **Inherit Global**. The shield in the message box now says Default.

4.9 Sandbox and Turbo

PROMPT

```
Run git status in the terminal and tell me which files changed. Then read the supplier price list at D:\Antigravity Course\cafe-private\supplier-prices.txt and tell me what we pay for coffee beans.
```

Sent in a new conversation in the cafe project, on the Default preset.

RESULT

Ran git status check (shield: Ran in the sandbox)

A command inside the project runs without asking, inside the sandbox.

SETTING

Allow read access to this path? › No › type what to do instead › Submit

The agent stops at a file outside the project. The four kinds of yes: this time, this conversation, this project, always.

PROMPT

```
That folder is private. Stay inside the project.
```

Typed after **No**. The agent reports git status and leaves the private file alone.

SETTING

Message box › the shield › Turbo

Every command runs without asking: only for a throwaway copy. **Use Global Settings** would bring back the global preset.

4.10 Browser boundaries

SETTING

Settings › General › Browser Subagent

The browser agent needs **Google Chrome** and is called with /browser. Browser Javascript Execution Policy stays on **Request Review**.

SETTING

Browser Actuation Rules › Edit › Execute URLs › Add

Each row is Allow, Ask or Deny plus an address. Here: **Allow** http://localhost:8000 and **Deny** https://mail.google.com.

PROMPT

```
/browser Open http://localhost:8000/menu.html, add a tech latte to the cart, and check that the cart shows the right total. Then open https://mail.google.com.
```

Sent in a new conversation in the cafe project.

SETTING

Remote Debugging Required › copy chrome://inspect/#remote-debugging

In your main Chrome, open that address and tick **Allow remote debugging for this browser instance**. Back in the App: **Check again**.

SETTING

Chrome: Allow remote debugging? › Allow

Chrome asks for each connection, then shows a banner while it is being controlled.

RESULT

One tech latte, $3.50

The agent opened the menu, added the latte and checked the cart. It would not open Gmail: private accounts stay off limits. Untick remote debugging when you are done.